

SOC Analyst Playbook

Operations, triage & incident response for the Wazuh home-SOC lab

Security Lab

2026-07-06

Table of Contents

SOC Analyst Playbook.....	1
1. Purpose & how to use this playbook.....	2
2. The SOC operating model.....	2
3. Severity & prioritization	2
4. The triage loop (run on every alert).....	3
5. Investigation workflow (Tier 2)	4
6. Containment, eradication & recovery.....	4
7. Scenario playbooks.....	5
8. Proactive threat hunting (Tier 3)	6
9. Shift operations & health checks	6
10. Incident documentation template.....	7
11. Detection tuning & feedback loop.....	7
12. Quick reference.....	8

SOC Analyst Playbook

Operations, triage & incident response for a Wazuh + Sysmon + Shuffle SOC

Scope Wazuh SIEM/XDR, Windows 11 endpoint (agent + Sysmon), Metasploitable2 (syslog), Shuffle SOAR

Audience SOC analysts (Tier 1–3), lab operators

Version 1.0 · 2026-07-06

Sensitive values are shown as placeholders (<MANAGER_IP>, <METASPLOITABLE_IP>, etc.). Companion references: **DQL Query Language Guide** and **Searching Metasploitable Events**.

1. Purpose & how to use this playbook

This playbook turns raw Wazuh alerts into a repeatable analyst workflow: how to **triage**, **investigate**, **contain**, and **document** security events in this lab, plus ready-to-run **scenario playbooks** for the alert types this environment produces. It is written to mirror a real SOC so the habits transfer to production.

Use it two ways:

- **Reactive** — an alert fires; follow *Section 4 (triage loop)*, then jump to the matching *Section 7 scenario playbook*.
- **Proactive** — during a quiet shift, run the *Section 8 threat-hunting* queries and the *daily/weekly operational checks* in Section 9.

2. The SOC operating model

2.1 Analyst tiers

Tier	Role	Typical actions
Tier 1	Triage / monitoring	Watch the alert queue, classify true/false positive, gather context, escalate
Tier 2	Investigation / IR	Deep investigation, scope the incident, contain, coordinate response
Tier 3	Threat hunting / engineering	Proactive hunts, detection tuning, new rules, forensics, SOAR automation

In a one-person home lab you play all three — but keeping the *hats* separate keeps the discipline: triage first, investigate second, hunt when idle.

2.2 Core principles

- **Every alert gets a disposition.** Never leave an alert unactioned: it's a True Positive, False Positive, or Benign True Positive (real but authorized).
- **Document as you go.** If it isn't written down, it didn't happen. Use the Section 10 template.
- **Preserve before you change.** Capture evidence (alert JSON, logs, hashes) before containment.
- **Assume nothing from a single alert.** Corroborate across data sources (Sysmon + Security log + FIM + network).

3. Severity & prioritization

Wazuh scores every rule 0–15. Map that to analyst priority and response time (lab SLAs — tighten for production).

Wazuh level	Severity	Priority	Target response	Examples
15+	Critical	P1	Immediate	Successful exploit, ransomware behavior, rootkit
12–14	High	P2	< 30 min	Repeated brute force success, credential dumping, malware verdict
7–11	Medium	P3	< 4 h	Failed brute force, suspicious process, policy violation, new critical file
4–6	Low	P4	Next shift	Informational auth, service restarts, benign config changes
0–3	Info	P5	As time allows	Routine logs, successful logins

Prioritize by more than level. Bump priority up when an alert involves: an internet-facing or crown-jewel asset, credential access, multiple correlated alerts on one host, or activity outside business hours. In this lab, **Metasploitable2 is intentionally vulnerable** — treat confirmed exploitation there as a *contained training event*, not a production P1, but still investigate it fully for practice.

4. The triage loop (run on every alert)

A disciplined 6-step loop. Aim to complete Tier-1 triage in minutes.

1. **Observe** — read the alert: rule id, level, description, MITRE mapping, source (agent or <METASPLOITABLE_IP>), timestamp. (Threat Hunting → Events; see the DQL guide.)
2. **Scope** — is this one event or many? Pivot on the host and the source/user: agent.name: "<WINDOWS_HOSTNAME>" or data.srcip: "<METASPLOITABLE_IP>" over a widening time window.
3. **Corroborate** — pull the supporting telemetry: Sysmon process tree (Event ID 1), parent process, network connections (Event ID 3), the raw full_log, FIM changes, VirusTotal verdict.
4. **Classify** — assign a disposition:
 - **True Positive (TP)** — real malicious/unauthorized activity → go to the scenario playbook.
 - **Benign True Positive (BTP)** — real but authorized (admin action, known scan) → note and close; consider a tuning exception.
 - **False Positive (FP)** — rule misfired → close and open a *tuning* item (Section 11).
5. **Act** — contain if TP (Section 6), or close with a reason. Trigger/attach the Shuffle workflow where applicable.
6. **Document** — record disposition, evidence, and actions using the Section 10 template.

Rule of thumb: **if you can't disposition it, escalate it**. An undispositioned alert is an open risk.

5. Investigation workflow (Tier 2)

When triage yields a True Positive, expand into a structured investigation.

1. **Establish the timeline.** Sort events for the host by timestamp. Identify the earliest suspicious event ("first seen") and the latest.
 2. **Reconstruct the process story** (Windows): use Sysmon Event ID 1 to walk the process tree — what launched what, command lines, hashes, signed vs unsigned. Look for powershell.exe, cmd.exe, wscript, LOLBins, encoded commands.
 3. **Check for persistence.** FIM registry keys (Run/RunOnce/Services/Winlogon), new scheduled tasks/services, new autorun files.
 4. **Check for credential access.** LSASS access (Sysmon EID 10), SAM dump (reg save hklm\sam), repeated auth failures then a success.
 5. **Check for lateral movement / C2.** Sysmon network events (EID 3), unusual destinations, new listening ports, SMB/RDP.
 6. **Determine scope & impact.** Which hosts, accounts, data are affected? Is it contained to the lab segment?
 7. **Decide response.** Contain now vs monitor to learn more (in a lab, prefer to observe the full attack chain, then contain — it's a learning environment).
-

6. Containment, eradication & recovery

Phase	Windows endpoint	Metasploitable2 (lab)	Manager/SOAR
Contain	Isolate host (disable NIC / firewall block), disable compromised account, kill malicious process	Snapshot the VM; it's already network-isolated to syslog-only	Shuffle workflow: block IP / disable AD account / notify
Eradicate	Remove malware/persistence, reset credentials, patch the entry vector	Revert to clean snapshot	Update rules/IOCs
Recover	Restore from known-good, re-enable access, monitor closely	Redeploy from base image	Confirm alerts return to baseline
Lessons learned	Post-incident review, tune detections, update this playbook	Document the attack path used	Add/adjust detection rule; add SOAR step

Automated containment via Shuffle: for high-confidence TPs, the Wazuh→Shuffle integration can fire a workflow (e.g. disable the offending AD user on a SAM-dump alert, or notify on brute force). Keep automation scoped and reversible; never auto-disable break-glass/admin accounts.

7. Scenario playbooks

Each scenario: **trigger** → **triage** → **investigate** → **contain** → **close**. Query snippets are DQL (see the DQL guide).

7.1 SSH brute force against Metasploitable2

- **Trigger:** repeated authentication failure / sshd alerts from <METASPLOITABLE_IP> (Wazuh rule groups authentication_failed, syslog, sshd).
- **Triage:** data.srcip:"<METASPLOITABLE_IP>" AND rule.groups:"authentication_failures" — count failures, note source of the attempts and target accounts.
- **Escalate the concern** if failures are followed by a **success** (rule.groups:"authentication_success") = possible credential compromise.
- **Contain:** snapshot the VM; if this were production, block the source IP (Shuffle firewall-drop) and reset the account.
- **Close:** TP (training) — record the attack source, count, and whether it succeeded.

7.2 Windows failed logons / password spraying

- **Trigger:** Windows Security 4625 (failed logon) bursts; rule groups authentication_failed, win_authentication.
- **Triage:** agent.name:"<WINDOWS_HOSTNAME>" AND data.win.system.eventID:"4625" — group by target user and source. Many users / few attempts each = spraying; one user / many = brute force.
- **Investigate:** look for a subsequent 4624 (success) from the same source.
- **Contain:** disable the account, force password reset, block source.

7.3 Suspicious process / LOLBin (Sysmon)

- **Trigger:** Sysmon EID 1 with encoded PowerShell, certutil, mshta, rundll32 with odd args, or a script interpreter spawning from Office.
- **Triage:** data.win.system.eventID:"1" AND data.win.eventdata.commandLine:*-enc* (or the LOLBin name). Inspect the **parent process** and command line.
- **Investigate:** hash the image, check VirusTotal, walk the process tree, look for follow-on network (EID 3) or file writes.
- **Contain:** kill the process, isolate, remove persistence.

7.4 Malware / EICAR (FIM + VirusTotal)

- **Trigger:** FIM “file added” on a monitored path followed by a VirusTotal malicious verdict (rule groups syscheck, virustotal).
- **Triage:** rule.groups:"virustotal" AND data.virustotal.malicious:>0 — identify the file, path, and detection ratio.
- **Investigate:** where did the file come from (browser download, share), was it executed (Sysmon EID 1 on that hash)?
- **Contain:** quarantine/delete the file, block the hash, scan the host.

- **Note:** Windows Defender may quarantine before FIM hashes it — a Defender detection (windows_defender group) may be the only sign (see the runbook's EICAR gotcha).

7.5 File integrity / persistence change

- **Trigger:** FIM change on C:\Windows\System32..., a Run key, or a service registry key.
- **Triage:** rule.groups:"syscheck" AND syscheck.path:*CurrentVersion\\Run*.
- **Investigate:** who/what changed it (correlate with Sysmon around the same timestamp), is it a known-good update or unauthorized persistence?
- **Contain:** revert the change, remove the persistence, find the initial access.

7.6 Credential dumping (SAM / LSASS)

- **Trigger:** reg save hklm\sam (Wazuh rule 92026) or LSASS access (Sysmon EID 10).
- **Triage:** rule.id:"92026" or data.win.system.eventID:"10" AND data.win.eventdata.targetImage:*lsass.exe*.
- **Contain:** this is high severity — isolate immediately, reset affected credentials. The Wazuh→Shuffle SAM-dump workflow can auto-disable the offending account.

8. Proactive threat hunting (Tier 3)

Run these when the queue is quiet. Full query syntax is in the DQL guide.

- **New/rare processes:** review Sysmon EID 1 for images never seen before this week.
- **Encoded / obfuscated commands:** data.win.eventdata.commandLine:(*-enc* OR *FromBase64String* OR *IEX*).
- **Outbound to rare destinations:** Sysmon EID 3 to non-standard ports/IPs.
- **Authentication anomalies:** successful logon after a burst of failures; logons outside normal hours.
- **Persistence sweep:** all FIM changes to Run keys / services / scheduled tasks in the last 7 days.
- **Metasploitable baseline:** review what the vulnerable box is doing normally so real attacks stand out (see the Metasploitable search guide).

Log every hunt (hypothesis, query, result) even when you find nothing — negative results are data.

9. Shift operations & health checks

Start of shift

- Confirm all sources are reporting: Agents summary shows the Windows agent **Active**; Metasploitable syslog arriving (data.srcip:"<METASPLOITABLE_IP>" in the last hour).
- Review the overnight alert counts (Critical/High first).

- Check manager health: `systemctl is-active wazuh-indexer wazuh-manager wazuh-dashboard`.

During shift

- Work the queue by priority (P1→P4). Disposition every alert.
- Keep an eye on repeat offenders and correlated alerts.

End of shift

- Hand off open investigations with notes.
- Note any detection gaps or noisy rules for tuning.

Weekly

- Tune the top 10 noisiest rules.
- Review SCA/CIS score drift and vulnerability findings.
- Verify backups of rules/configs.

10. Incident documentation template

Incident ID: INC-YYYYMMDD-NN
 Analyst: <name> Date/Time (UTC):
 Source alert(s): rule.id / level / description
 Affected asset(s): <hostname / IP>
 Disposition: TP | BTP | FP
 Severity/Priority: Critical/High/Med/Low · P1-P5
 Timeline:
 - first seen:
 - key events:
 - contained:
 Summary (what happened):
 Evidence (alert JSON, hashes, log excerpts, screenshots):
 MITRE ATT&CK (tactic / technique):
 Actions taken (triage, containment, eradication, recovery):
 Root cause / entry vector:
 Follow-ups (tuning, new rule, SOAR step):
 Status: Open | Contained | Closed

11. Detection tuning & feedback loop

- **False positive** → don't just close it; reduce future noise. Add a narrow exception, adjust the rule level, or write an override rule (custom rules use IDs **100000+**). Document why.
- **Missed detection (gap)** → write/adjust a rule; add the log source; validate with `wazuh-logtest`.
- **Repeatable response** → promote it into a Shuffle workflow.

- Feed every lesson back into this playbook and the runbook so the SOC gets smarter over time.

12. Quick reference

Need	Where
Alert feed	Dashboard → Threat Hunting → Events
Filter by severity	<code>rule.level >= 12</code> (high/critical)
Windows host alerts	<code>agent.name:"<WINDOWS_HOSTNAME>"</code>
Metasploitable alerts	<code>data.srcip:"<METASPLOITABLE_IP>"</code> (agentless — see its guide)
MITRE view	Dashboard → MITRE ATT&CK
Malware verdicts	Dashboard → Malware Detection
File changes	Dashboard → File Integrity Monitoring
Manager health	<code>systemctl is-active wazuh-indexer wazuh-manager wazuh-dashboard</code>
Automated response	Shuffle workflow (Wazuh→Shuffle integration)

End of playbook.